

PaperPipe v3.0: Stats Verification Agent

기술 사양서 (Final Version)

문서 버전: 4.1.0 (FINAL - Release Candidate 1)

대상: AI Engineer, Backend Developer, QA Team

목표: 과학 논문의 통계적 무결성을 검증하기 위한 자율 에이전트(Autonomous Agent) 구축

1. 개요 (Executive Summary)

Stats Verification Agent는 논문에 보고된 통계 수치(test statistic, df, p-value)를 추출하고, 원본 데이터(Table)를 기반으로 이를 ****재계산(Recalculation)****하여 검증하는 시스템입니다. 단순한 텍스트 비교가 아닌, Python 실행 환경을 활용한 **Code-Based Reasoning**을 수행합니다.

핵심 설계 철학

- Code-First Parsing:** 복잡한 과학 표(Table)를 텍스트가 아닌 pandas 코드로 변환하여 해석 정확도 극대화.
- Stateful Orchestration:** LangGraph를 도입하여 에이전트의 순환적 추론(ReAct)과 여러 복구(Reflexion) 과정을 상태 머신으로 관리.
- Algorithmic Verification & Precision:** 단순 p 값 비교를 넘어 α 수준의 가변성 처리, 자유도(df)의 다중 구조화, 그리고 모든 통계량(t , F , χ^2 등)의 반올림 오차(Interval-Consistency)까지 고려한 정밀 검증.
- Hardened Security:** 호스트 시스템 보호를 위해 **Docker MicroVM Sandbox** 환경에서 실행하되, 명확한 읽기/쓰기(RO/RW) 마운트 권한 분리.
- Asynchronous Architecture:** Verify 단계의 병목과 타임아웃을 방지하기 위한 워커(Worker) 분리 및 마스터 스펙 표준을 준수하는 SSE(Server-Sent Events) 기반 진행률 스트리밍.

2. 시스템 아키텍처 (System Architecture)

시스템은 **Controller(LangGraph)**, **Reasoning Engine(LLM)**, **Execution Environment(Sandbox)** 세 가지 핵심 요소로 구성됩니다.

코드 스니펫

graph TD

```
UserQuery(Input: DocumentArtifact & ClaimSet) --> Graph(LangGraph Controller)
```

```
subgraph "Reasoning Loop (Async Worker)"
  Graph --> NodeParse(Table-to-Pandas / Check Input Quality)
  NodeParse --> NodeExtract(Extract Reported Stats)
  NodeExtract --> NodePlan(Stat Test Planning)
  NodePlan --> NodeCode(Code Generation)
  NodeCode --> NodeReflect(Reflexion & Fix)
end
```

```
subgraph "Execution Sandbox (Docker MicroVM)"
  NodeCode -- Exec Code --> PythonRuntime
  PythonRuntime -- Stdout/Stderr --> NodeCode
end
```

```
Graph --> Validator(Pydantic Validation & Evidence Mapping)
Validator --> Output(StatsReport JSON)
```

3. 데이터 계약 (Data Contracts via Pydantic)

입출력의 엄격한 통제를 위해 Pydantic을 사용합니다. 마스터 스펙과의 완벽한 정합성을 위해 StatsReport는 개별 checks 중심의 독립적인 레코드로 설계되었습니다.

3.1 근거 추적 스키마 (EvidenceSpan)

HITL(Human-In-The-Loop) 검수 비용을 낮추기 위해 rationale을 필수로 강제하며, 프론트엔드 연동 오류를 막기 위해 0-index 기반의 페이지 표준을 준수합니다.

Python

```
from pydantic import BaseModel, Field, field_validator
from typing import Optional, Literal, List
```

```
class EvidenceSpan(BaseModel):
    """추출된 데이터의 논문 내 근거 위치"""
    page: int = Field(..., description="0-indexed PDF 원본 페이지 번호 (0부터 시작)")
    chunk_id: str = Field(..., description="DocumentArtifact에 정의된 표준 chunk_id")
    char_start: Optional[int] = Field(None, description="체크 내 시작 오프셋")
    char_end: Optional[int] = Field(None, description="체크 내 종료 오프셋")
    raw_text: str = Field(..., description="추출된 원문 텍스트 또는 표의 캡션")
    quote: Optional[str] = Field(None, description="원문 발췌 (25단어 이하 권장)")
```

```
rationale: str = Field(..., description="이 텍스트가 근거로 채택된 이유 (1~2문장, 필수)")
```

3.2 검증 결과 및 레포트 스키마 (StatsReport & StatCheckEntry)

각 StatCheckEntry는 자급자족(self-contained) 형태로 구성되어 코드, 로그, 판정 결과를 모두 내재화합니다.

Python

```
class VerificationStatus(str, Enum):
    VERIFIED = "verified"          # 표 재계산 일치 AND 텍스트 보고 수치 근거 완벽 확보
    PARTIALLY_VERIFIED = "partially_verified" # 표 재계산 일치하나 텍스트 근거 불충분 (Internal Consistency 성립)
    INCONSISTENT = "inconsistent"   # 재계산 수치와 보고 수치 불일치
    UNVERIFIABLE = "unverifiable"  # 데이터 부족 또는 파싱 실패로 검증 불가

class StatCheckEntry(BaseModel):
    """개별 통계 검증 항목 (자급자족형 레코드)"""
    check_id: str = Field(..., description="고유 검증 ID (ClaimSet의 claim_id와 1:1 매핑)")
    hypothesis: Optional[str] = Field(None, description="검증 대상 가설")
    method: Optional[str] = Field(None, description="분석 방법론")
    test_type: str = Field(..., description="수행된 통계 테스트 (예: t-test, ANOVA)")

    # 보고된 값 (Extraction)
    reported_stat: Optional[float] = Field(None)
    reported_df_raw: Optional[str] = Field(None)
    reported_df_tuple: Optional[List[float]] = Field(None, description="파싱된 n개의 자유도 배열 (복잡한 모형 대응)")
    df_parse_status: str = Field(..., description="'success', 'failed' 등 파싱 상태")
    reported_p: Optional[str] = Field(None)
    evidence: List = Field(default_factory=list)

    # 알파(유의수준) 처리
    alpha_used: float = Field(0.05, description="적용된 유의 수준")
    alpha_evidence: Optional = Field(None, description="0.05가 아닐 경우 명시적 근거")

    # 재계산된 값 (Recalculation)
    computed_p: Optional[float] = Field(None)

    # Interval-Consistency (반올림 오차 보정)
    interval_consistency_applied_to: List[str] = Field(default_factory=list, description="적용된 통계량 (예: ['t', 'F', 'chi2'])")
```

```
interval_consistency_skip_reason: Optional[str] = Field(None, description="미적용 사유 (예: '지원하지 않는 분포')")
```

```
# 판정 결과 및 실행 로그
verification_status: VerificationStatus = Field(...)
decision_error: bool = Field(False, description="유의성 결론 번복 여부")
notes: Optional[str] = Field(None, description="불일치 원인 분류 코드(예: ERR_ROUNDING) 및 자연어 설명")
code: str = Field(..., description="검증에 사용된 Python 코드 전문")
outputs: dict = Field(..., description="{ 'summary': '실행 요약 해시 또는 에러 메시지', 'log_path': '전체 로그 파일 경로' }")
```

```
class StatsReport(BaseModel):
    """최종 산출물 (저장 대상 - 마스터 스펙 완전 준수)"""
    schema_version: str = Field("1.2")
    paper_id: str
    run_id: str
    input_tables_used: List[str] = Field(default_factory=list, description="분석에 사용된 DocumentArtifact table_id 목록")
    sandbox: dict = Field(..., description="샌드박스 환경 메타데이터 (예: 도커 이미지, 실행 시간)")
    checks: List
```

3.3 출력 및 저장 정책 (Storage Policy: OOM 방지)

- **JSON** 파일 경로: `storage/artifacts/{paper_id}/{run_id}/stats_report.json`
- 로그 파일 분리: 샌드박스 실행 로그(`stdout/stderr`)가 길어져 JSON 크기가 폭발하고 동기화(`Sync`)가 느려지는 현상을 막기 위해, 원본 로그는 `storage/sandbox/{job_id}/execution_{check_id}.log`에 텍스트 파일로 저장합니다.
- **StatsReport**의 **outputs**: JSON 스키마에는 에러의 요지나 해시, 그리고 원본 로그 파일의 경로만 저장합니다.

4. 상세 워크플로우 및 상태 관리

4.1 비동기 워커 및 SSE 이벤트 계약 준수 (Stage Mapping)

Verify 단계는 비동기 환경에서 실행되며, 마스터 스펙의 SSE 이벤트 규격을 파괴하지 않고 세부 상태를 전송하기 위해 `stage`를 메이저/마이너로 이원화합니다.

SSE Payload Example:

JSON

```
{
  "job_id": "job_12345",
  "progress": 45,
  "stage": "verify",
  "details": {
    "stage_minor": "parse_table",
    "message": "Parsing complex multi-index tables into pandas dataframe."
  }
}
```

- stage는 반드시 마스터 스펙의 고정된 대분류(verify, completed, error 등)를 따릅니다.
- details.stage_minor에 parse_table, extract_reported_stats, plan, execute, reflect 등의 세부 진행 상황을 기록하여 UI에 제공합니다.

4.2 Degradе Mode (표 입력 품질 요건)

애매한 시각적 기준(2/3 충족 등) 대신 마스터 스펙의 DocumentArtifact 계약을 기반으로 강등(Degrade) 여부를 깔끔하게 판정합니다.

- 판단 기준: DocumentArtifact.tables[i]의 extraction_status == "ok" 이고, dataframe_json(또는 html_table) 데이터가 존재하는 경우에만 Pandas 기반의 심층 재계산을 시도합니다.
- 강등 동작: 위 조건을 만족하지 못하면 조기 실패(Fail-fast) 처리하고, 본문 텍스트에 보고된 (t, df, p) 수치만을 바탕으로 내부 일관성(Internal Consistency)만 체크하는 모드로 전환합니다. 이 경우 최고 상태는 PARTIALLY_VERIFIED로 제한됩니다.

5. 엄격한 검증 알고리즘 (Strict Verification Logic)

5.1 Verification Status 승격 기준 및 UI 연계

표 기반 재계산은 전처리/보정 등에 대한 가정이 개입되므로 VERIFIED 상태를 남발해서는 안 됩니다.

1. **PARTIALLY_VERIFIED (기본값):** 표 기반 재계산 결과가 내부적으로 일치함(Internal Consistency 성립). 주의: 완전한 VERIFIED가 아니더라도 이 상태는 논문의 내적 정합성을 증명하므로 매우 가치 있습니다. UI 노출 시, notes 필드에 원인 분류 코드(예: ERR_TEXT_EVIDENCE_MISSING)와 "표 데이터로는 일치하나 본문 근거가 불충분함"과 같은 설명을 1줄로 제공하여 신뢰도를 높입니다.
2. **VERIFIED (승격):** 표 재계산 결과가 일치하며, 동시에 reported_stat, df, p 값이 모두 EvidenceSpan을 통해 본문에서 **완벽하게 그라운드(Grounded)**된 경우에만 승격됩니다.

5.2 Interval-Consistency (반올림 오차 보정 확장)

단순한 t-통계량뿐만 아니라, 분산 분석의 F값, χ^2 값, r값 등 다른 통계량에 대해서도 반올림 오차 구간을 검증해야 편향을 막을 수 있습니다.

- 에이전트는 반올림된 통계량(예: 2.13)의 하한값(2.125)과 상한값(2.135) 구간에서 생성되는 p-value의 Interval을 도출합니다.
- `interval_consistency_applied_to`에 ["t", "F"] 형태로 적용된 통계량을 명시하여 투명성을 확보합니다.

5.3 Alpha 가변성 및 Decision Error

결정 오류(Decision Error)는 계산된 결과가 저자의 유의성 결론을 뒤집을 때 발생합니다.¹

- `alpha_used`를 식별하고, 명확한 다중 비교 보정 근거가 없으면 0.05를 기본값으로 하되 notes에 "근거 없음"을 표기합니다.
- $p_{comp} < \alpha_{used}$ 와 $p_{rep} < \alpha_{used}$ 의 방향성이 다를 때만 `decision_error = True`로 처리됩니다.

6. 보안 및 인프라 (Security Infrastructure)

6.1 Docker Sandbox 마운트 규칙 (RW / RO 분리)

샌드박스 실행 시 호스트 보호와 Pandas의 정상적인 구동을 동시에 만족시키기 위해 명확한 볼륨 마운트 권한 분리를 스펙으로 강제합니다.

- **Root File System:** 컨테이너 커널 및 시스템 폴더 보호를 위해 반드시 읽기 전용(`--read-only=True`)으로 마운트합니다.
- **Work Directory (Writable):** Pandas 임시 파일, SQLite, 그리고 `execution_logs` 파일 덤프를 위해 오직 `/work` 경로만 쓰기를 허용합니다.
- 경로 매핑: 호스트의 `storage/sandbox/{job_id}/`를 컨테이너의 `/work`에 바인드 마운트(`rw`)합니다.

Python

```
import docker
client = docker.from_env()

container = client.containers.run(
    "paperpipe/stats-sandbox:latest",
    command=["python", "/work/script.py"],
    network_mode="none",          # 외부 접속 차단
    mem_limit="512m",
```

```
read_only=True,          # [필수] 루트 파일 시스템 RO
volumes={
    f"/host/storage/sandbox/{job_id}"/": {'bind': '/work', 'mode': 'rw'} # [필수] 특정 폴더만 RW
},
working_dir="/work",
user="1000:1000",        # 비특권 사용자
remove=True
)
```

7. 데이터 파이프라인 및 부스팅 오염 방지 (Phase 0)

추후 교사-학생(Teacher-Student) 모델 부스팅 파이프라인에서 훈련 데이터의 무결성을 유지하기 위한 최우선 하드 룰(Teacher Rubric)입니다.

- **Evidence** 강제: StatCheckEntry에 evidence (특히 rationale과 page 정보)가 하나라도 누락되거나 부실한 경우, 교사 모델의 검증 루브릭은 해당 항목의 학습 라벨을 무조건 `accepted=false`로 처리해야 합니다. 어떠한 경우에도 명시적인 증거와 이유(rationale)가 없는 검증 결과를 `true`로 승인해서는 안 됩니다.
- 교차 검증: 기본적으로 모든 자동 검증 결과는 `accepted=false`로 시작하며, Cross-teacher PASS 조건을 완벽히 통과한 경우에만 훈련 데이터셋(Phase 1)으로 승격됩니다.

참고 자료

1. Manual statcheck 1.3.0 - RPubs, 2월 11, 2026에 액세스,
<https://rpubs.com/michelenuijten/statcheckmanual>